

Manual of Experiments for

CONTROL ENGINEERING LABORATORY

(Course Material for EE208)

2026

Contents

1. Dynamic response of transfer functions on MATLAB platform.
2. Dynamic response studies on a V/f controlled induction motor drive.
3. Controller design on MATLAB platform using analog root loci.
4. Position control by a servomotor actuator.
5. Controller design on MATLAB platform by analog frequency response.
6. Dynamics of thermal systems under P/PI/PID control, as studied on Siemens SCADA.
7. Controller design on MATLAB platform using discrete root loci.
8. Dynamics of thermal systems under P/PI/PID control, as studied on Siemens SCADA.
9. Controller design on MATLAB platform by discrete frequency response.
10. Performance of a stepper motor as servo and drive actuator.
11. State feedback controller design on MATLAB platform.
12. Nonlinear system dynamics on Simulink for different Lyapunov control designs.

Experiment 1

Dynamic response of transfer functions on MATLAB platform.

Background:

The most fundamental and time tested approach for design and analysis in *systems and control* technology, is in terms of *system output dynamics over time* in response to *standard inputs*.

In case of strictly linear systems, such studies can be conveniently conducted in terms of *transfer functions* or *transfer matrix* (depending on whether the system is SISO or MIMO, respectively), either in *analog* or *digital domain*.

Further, from theory we know that several useful design and analytical *performance measures* have evolved from *dynamic response* concepts, such as ***peak overshoot***, ***peak time***, ***delay time***, ***settling time***, etc.

However, the standard formulae for each of these metrics assumes a second order response of the system. If the true transfer function of the system is of order higher than two, the standard second order formulae are inaccurate to varying extent.

Objective:

The project experimentally investigates the inaccuracy of *second order performance metrics* for a system that is of fourth order with two zeros.

Tutorial: (to be covered in not more than an hour)

In the MATLAB platform, go through the procedural steps as described for:

- *Control system modeling with model objects:*
<https://www.mathworks.com/help/control/ug/control-system-modeling-with-model-objects.html>
- *Connecting models:*
<https://www.mathworks.com/help/control/examples/connecting-models.html>
- *Creating discrete-time models:*
<https://www.mathworks.com/help/control/examples/creating-discrete-time-models.html>

Elements of the software for familiarisation:

- a. **Models:** Continuous-time zero/pole/gain model; sample time; discrete-time transfer function.

- b. **Functions:** append, connect, feedback, frd, grid, isdt, parallel, pid, series, ss, step, stepplot, sumblk, tf, zpk.
- c. **Precedence rule:** frd > ss > zpk > tf > pid.

Project: (to be covered in about two lab hours or with additional study hours, if required.)

The fourth order open loop transfer function with zeros, is provided to you, with the open loop gain parameter K in the range $0 < K < \infty$.

If the loop is closed through unity negative feedback, the CLTF will take us through a range of dynamic behaviour that are to be observed, analysed and discussed *exclusively through time domain simulations!*

For observations and discussions:

Complete a comparison table using the step responses for different values of the gain K , including

- the percentage overshoot,
- the time to first peak,
- the settling time, and
- the steady-state error.

The comparisons are between two readings for each of the above metrics:

- a. As measured and estimated from *on-screen step response plots*.
- b. As obtained by approximate *second order response formulae* as we did in the preceding lecture course. (These quantities are to be directly computed using functions available in Matlab, all of which use second order approximations).

Comparatively comment on the results, and explain the reason for the differences observed in response.

If the **OL poles of the third order OLTF are open to $\pm 20\%$ variation**, investigate the effect of this on the four quantities listed above.

You are expected to draw as detailed conclusions as possible, but all exclusively *in terms of the step response*.

Experiment 2

Dynamic response studies on a V/f controlled induction motor drive.

Background:

Control systems categorised as **electric drives** are essentially combination of a *motor*, a driving *power electronic converter*, and *control system blocks* dedicated to the latter. The motivation to develop and use such systems have evolved from the fact that any motor has some “default” characteristic torque/speed variation as decided by its electromagnetics, and these are not always satisfactory for every application. The power electronic converter allows us to manipulate the input electrical variables (voltage, current), thereby resulting in modified (hopefully, for the better in some way !) torque/speed variations.

The above is however easier said than done, and during the course of a drive development, various nonlinearities and linearities may have to be accommodated – sometimes eliminated and, on occasions, deliberately included to achieve some performance feature. As the overall system becomes progressively complex, dominant dynamics of the combination can change in unexpected ways – again sometimes by intention, and at others without !¹

It is easy to understand that for practically all examples of electric drives, the *response of speed* to electrical *output of the converter* (voltage or current) becomes most dominant of the dynamics. The reason is partly due to the fact that this is usually the slowest of all dynamics, and sustains for the longest time. Second, while it is invariably decided by the rotor inertia, other electromagnetic nonlinearities affect it seriously as well. Eddy current damping in the rotor core, and core saturation are two such example nonlinearities.

In the present experiment, we study a very common form of drive that works on three phase induction motors – the so called *V/f control*. Revisiting the equivalent circuit of the three phase induction motor, note that the stator back emf per phase is proportionate to a product of flux per pole and frequency. If the stator series branch is of relatively low impedance (as is a common design practice for motors of high efficiency), then at constant flux this amounts to maintaining the terminal voltage proportionate to the stator frequency.

The commercial Siemens *SINAMICS V20* drive provided to us creates a DC power source by rectifying single phase AC (diode based full bridge rectifier), with electrolytic capacitors (internal – these parts are enclosed and not available to you) to stabilise the DC bus voltage. The DC bus feeds power to a three phase inverter, whose output three phase AC voltage and frequency can be controlled as

- *Constant V/f* (up to rated output voltage V_{max}) so as to maintain the core flux just at the “knee point” of the saturation curve.
- Constant output voltage (once V_{max} is reached)

The specs for the overall drive are: input 230VAC , 50Hz , output 3 Phase 0-230VAC, 0-50Hz.

¹ For those interested, you have a complete elective course on the subject of *electric drives* that is made available to you in a subsequent semester.

The motor connected is rated at 0.5hp, 1440rpm, three phase, 230VAC, 50Hz, 1.7A. On its shaft you will observe an electromagnetic *proximity sensor* that feeds an electrical signal proportionate to the shaft speed to the drive controller, so that you get to see the rpm on a panel display.

Objective:

- a. To study various aspects of steady state performance by the Siemens V20 V/f drive on a three phase induction motor.
- b. To evaluate the dominant speed time constant under different steady state V/f operating points of the drive.

Tutorial:

You will get many application websites on this drive, but the basic one is:

<https://www.siemens.com/global/en/products/drives/sinamics/low-voltage-converters/standard-performance-frequency-converter/sinamics-v20.html>

For those interested to know more, try this one:

https://www.pccweb.com/wp-content/uploads/2015/08/Siemens_MCD_V20_Getting_Started_Manual.pdf

Project:

Objective “a”:

General instructions:

- ◆ Provide the required single phase 230V AC , 50Hz power supply to the panel .
- ◆ Before starting the operation of AC drive trainer make sure that initial settings are done on VFD (the VFD is Siemens’ commercial name for the *Voltage-Frequency-Display*).
- ◆ Power on the *mains*, the *AC drive*, and the *rpm indicator*.

Operations for the Manual mode:

- ◆ To operate the motor in *Hand* mode (which is a manual control mode), long press both **multifunction (M)** and **OK** keys simultaneously for a little less than about 2s. The symbol of a *hand* will be displayed on the screen, indicating the *Hand* mode.
- ◆ If the same keys (**M** and **OK**) are pressed again in the same *Hand* mode, the drive enters the *Jog* mode, which is indicated by a *blinking hand* symbol. In the *Jog* mode, start and stop function as usual, but the motor always works at a constant inverter frequency (hence, synchronous speed).
- ◆ Long press (more than about 2s) the **multifunction (M)** button to get out of the *setting* to *home* screen at any point.
- ◆ The green (**1**) and red (**0**) buttons on the VFD can be used to respectively start and stop the motor at any time.
- ◆ The frequency (hence synchronous speed) can be raised or lowered respectively by the UP and DOWN keys on the VFD; the sequence of steps being as follows:
 - i. Once at HOME screen, press the button marked OK. (The voltage applied to the motor by the drive is displayed.)

- ii. Press the OK button once again. (The current fed to the motor by the drive is now displayed.)
- iii. Press the OK button once again. (Displays the control mode DC323.)
- iv. Once on DC323, immediately press UP/DOWN buttons to modify frequency of the AC fed to the motor by the drive.

Operations for the Automatic mode:

- ◆ Short press (less than 2s) the **multifunction (M)** button to flip between sequentially appearing successive screens or the *settings* screens.
- ◆ *Settings* can be navigated through by the UP and DOWN keys.
- ◆ Long press (more than 2s) the **multifunction (M)** button to reach the *Home* screen. A second short press (a little less than 2s) takes you to the **Cn007** macro, which decides operation in *Automatic* mode.
- ◆ Once at the **Cn007** macro, move to option setting **P1000**.
 - Setting **P1000** to “2” allows the option of *continuous frequency control* (hence synchronous speed control) by the potentiometer knob provided.
 - Setting **P1000** to “10” allows the option of *manual frequency control* (hence synchronous speed control) as in the *Manual* mode described above.
 - Setting **P1000** to “12” allows a choice of options between *continuous frequency control* and *manual frequency control* (synchronous speed control accordingly). In this setting, control by the potentiometer knob provided has a high priority over the latter. If the potentiometer is set to zero, the control automatically becomes as described for *Manual*.
- ◆ The *settings* screens can be exited by long pressing the **multifunction (M)** button.
- ◆ Once out of the *settings*,
 - Switch **D1** can be used to start (low position) or stop (raised position) the motor drive, and
 - Switch **D2** can be used to choose forward rotation (low position), and **D3** for reverse rotation (low position). (**Before changing between D2 and D3, always make sure that D1 is off, that is, the drive should be disconnected from the motor.**)

Note that all three switches are off in the raised position.

Objective “b”:

We now evaluate the dominant first order time constant of the drive/motor under different settings of the inverter output frequency (recall that *V/f control* is in operation as described at the end of the **Background** section).

- In *Automatic* mode (**Cn007** macro settings), set **P1000** to “2” (*continuous frequency control*).
- With **D1** off, switch **D2** or **D3** to choose motor direction.
- Set the potentiometer at some value of inverter frequency.
- Turn on **D1**. As the motor attains the expected steady speed, record different values of speed attained at different instances from **D1** switching (use the timer or stopwatch on your mobile for the timing). To get the complete response you may have to repeat this process several times.
- Obtain the *effective time constant* for the step startup of speed from the complete response recorded.
- Note that while the dominant dynamics are always first order (maybe second order sometimes !), the time constants at each frequency setting (*V/f* in operation) need not be the same !

For observations and discussions:

Objective “a”:

For each direction of rotation, tabulate:

- i. Inverter frequency
- ii. Inverter voltage (recall V/f control is in effect)
- iii. Internal DC link voltage (ideally this should be constant between the rectifier and inverter, but is it ?)
- iv. Inverter current (since the motor shaft is not loaded, the inverter only feeds the free running current to the motor, which should again be ideally constant and negligible).

Your discussion and comments should be in terms of the non-idealities observed and the internal aspects of the drive and motor that affect the results tabulated.

Objective “b”:

For each *direction of rotation*, *inverter frequency* and *inverter voltage* (V/f is in effect), tabulate the dominant time constant (or time constants) for speed dynamics.

Discuss the internal linearities and nonlinearities that affect the response. Further to be noted and discussed is the effect of different *voltage-frequency* settings as per V/f .

Experiment 3

Controller design on MATLAB platform using analog root loci.

Background:

We have had ample briefing on **CL pole plots** in the theory course particularly in the context of *time domain control designs*, so not much remains to be said about the concepts involved.

Here we first examine the power of MATLAB in implementation of the technique, followed by a realistic project on the same.

Objective:

The project examines *damping of oscillations* at *very high gains* for a linearised nonlinear system, with a single *critical parameter* that is susceptible to *significant variation* subject to point of linearisation.

Tutorial: (to be covered in not more than an hour)

In the MATLAB platform, go through the procedural steps as described for:

- *Root locus design:* <https://www.mathworks.com/help/control/ug/root-locus-design.html>

Elements of the software for familiarisation:

- App:** `controlSystemDesigner('rlocus','*')`
- Functions/keywords:** `rlocus; rlocusplot; getoptions;setoptions; sgrid; plotoptions; tf; zpk.`

Project: (to be covered in about two lab hours or with additional study hours, if required.)

The specified third order analog OLTF is to be operated in unity negative feedback closed loop with the *gain parameter to assume different positive values*. We are primarily interested to operate the system at high values of *open loop gain*; the focus being on *oscillation damping* at high gains.

What seems to complicate matters is a single real pole parameter “*a*”, which may assume values ranging from that of the single real zero, to infinity. Since the OLTF emerges from linearisation of a nonlinear system, the value of “*a*” depends strongly on the equilibrium about which the system is linearised.

For observations and discussions:

By detailed root locus studies and results plotted therefrom, analyse and discuss the impact of the parameter " a " (ranging between the open loop zero, to infinity) on the CLTF behaviour.

Your discussions should involve plots between CLTF damping component at close to Infinity values of K , against the OLTF pole parameter " a ".

Precisely stated, **exactly how does the parameter " a " influence root loci for the system ?**

Is stability an issue under any circumstance ?

Experiment 4

Position control by a servomotor actuator.

Background:

A servomotor is a ***closed-loop position controller*** that uses *position feedback* to control its motion and final position. The input to its control is a signal (either analog or digital) representing the *position reference* for the output shaft.

The motor is paired with some type of *encoder* to sense position and speed for feedback. In the simplest case, only the position is measured. The *measured position* of the output is compared to the *position reference* (can be for *regulation* or *tracking* applications). As the two position values approach each other, the error signal reduces to zero and the motor stops.

The very simplest servomotors use ***position-only sensing*** via a potentiometer and ***bang-bang control***² of their motor; the motor either rotates at rated speed to realise a change, or is stopped. This type of servomotor is not widely used in industrial motion control, but it forms the basis of the simple and cheap servos used for radio-controlled models.

More sophisticated servomotors use optical *rotary encoders* to measure the speed of the output shaft and a *variable-speed drive*. Both augmentations, usually in combination with a PID control algorithm, allow the servomotor to follow *position reference* in lesser response time and with greater precision.

Objective:

Performance evaluation of a Mitsubishi 230V AC servomotor, controlled by the Mitsubishi MR-JE-40AS servo controller.

Tutorial:

For those interested, details of the controller are available as:

<https://dl.mitsubishielectric.com/dl/fa/document/manual/servo/sh030128/sh030128j.pdf>

Project:

1. Power on the system.
2. Turn on the 24V power supply to the *graphic operation controller* by the ON switch.
3. Power the servomotor the MOTOR ON switch. **Position Control** will be displayed on the *human machine interface* screen.

² The term *bang-bang control* refers to a simple ON/OFF type control that switches depending on whether the output error is positive or negative. Very cheap to implement, the control is found to be useful in a large range of not-so-sensitive applications.

4. Press *right arrow* (→) to go to next screen. **Screen 2** will be displayed where we can set **Servomotor speed** by using function keys (F1-F5) followed by *Enter* (↵) key. The value of speed as sensed by the *photoelectric sensor* will be displayed on the same screen.
5. Direction of servo motor will be decided by **FWD/F2** and **REV/F4** keys.
6. **K1-K8** keys are used as '**FUNCTION KEYS**' as well as '**ACTION DISPLAY KEYS**'. Specifically, the motor can be stopped by **K3** key.
7. **Before using position mode always execute homing operation** by **K5** key.
8. Once the homing is done press *right arrow* (→) key to go to **Screen 3 (Position screen)**. The *position reference*, as well as the *current position* as sensed are displayed on this screen. will find set angle as well as current angular position. We may set the *position reference* by function keys (F1-F5) and press *Enter* (↵) key.
9. Press **K4** key to enable position mode. Direction of rotation can be selected by **F2 (FWD)** and **F4 (REV)** keys.
10. (Turn off all switches once the study is complete.)

Precautions: The speed limit and position limit of this servomotor are set at 2000 rpm and 360°, respectively.

For observations and discussions:

- In **Screen 2**, set different values of *reference speed* (within the specified limit), run the servomotor, and record the sensed value of *servomotor speed* by the bang-bang control implemented. Discuss and explain the percentage error trend that is observed across the range permissible.
- In **Screen 3**, set different values of *reference position* (within the specified limit), run the servomotor, and record the sensed value of *servomotor position* by the bang-bang control implemented. (Remember to precede each experiment by the homing as mentioned in #7 above.) Discuss and explain the percentage error trend that is observed across the range permissible.
- Discuss why it becomes essential to stay within the speed and position limits mentioned in the precautions as above.

Experiment 5

Controller design on MATLAB platform by analog frequency response.

Background:

Check out the advantages of *bandwidth* based **frequency domain designs**, as covered in your lecture course !

The depth of a submarine is governed by a third order open loop transfer function provided for the problem.

By default, the control is operated by closed loop *proportional control* with unity negative feedback such that:

- Steady state position error is 5% or less,
- 5% settling time for step response is 10s or less, and
- Peak overshoot to step response is 2% or less.

Objective:

The project examines effect of lead and/or lag compensation on a largely low pass, analog SISO system.

Tutorial: (to be covered in not more than an hour)

In the MATLAB platform, go through the procedural steps as described for:

- *Frequency-Domain Characteristics on Response Plots:*
<https://www.mathworks.com/help/control/ug/system-characteristics-on-response-plots.html>
- *Frequency response of a SISO system:*
<https://www.mathworks.com/help/control/ug/frequency-response-of-a-iso-system.html>
- *Frequency response of a MIMO system:*
<https://www.mathworks.com/help/control/ug/frequency-response-of-a-mimo-system.html>

Elements of the software for familiarisation:

a. **Functions/keywords:** bode; bodeoptions; bodeplot; freqresp; sigma; sigmaplot; tf; zpk.

b. **Tools:** Plot customization:
<https://www.mathworks.com/help/control/analysis-plot-customization.html>

Project: (to be covered in about two lab hours or with additional study hours, if required.)

Design a minimal order cascaded controller for submarine depth, so as to realise:

- Steady state position error of 10% or less,
- Phase margin of 60° or more,
- Gain margin of 10dB or more.

For observations and discussions:

Compare the frequency response based control design with the default control design, by checking the extent to which the design specifications for each design are met, or not met, by the alternative design.

By due consideration of all aspects, figure out which design is superior of the two, and why ?

Experiment 6 & 8

Dynamics of thermal systems under P/PI/PID control, as studied on Siemens SCADA.

Background:

In both Expts. 6 and 8, we set *Proportional* (P), *Proportional-Integral* (PI), and *Proportional-Integral-Derivative* (PID) controllers for given thermal systems using **Ziegler-Nichols** tuning rules. The performance of the systems with the controllers designed by you, is compared against *automated setting of corresponding controllers* by the implemented **Fuji Electric's PXF 5/9 Microcontrol X**.

Of the two experiments, **Expt. 6 uses a very slow oven** provided with a heater and an RTD thermal sensor of temperature, whose output is used as a feedback signal for the P/PI/PID. **The response of the oven in Expt. 8 is comparatively rapid**, though both are first order systems (recall the heater/temperature-sensor based furnace problem we did in the theory course ?).

The PLC based **Microcontrol X** can be set by us using buttons provided on the front panel of the controller block. But we have a better option in these experiments !

In each experiment, the Fuji **Microcontrol X** is connectable to Siemens **SIMATIC V15 SCADA**³ software running in real time on an appropriate host computer. The two are linked through a commonplace **RS 485** interface (**MODBUS-RTU** protocol), which allows you both a feel as well as the convenience of real time computerised data acquisition – the way most industries do today.

Some notable points regarding the apparatus for both experiments, before we begin the work:

1. The feedback controller operates on the furnace through a heater (and no cooler). Therefore, the P/PI/PID is only operational when the **set point** (**SV**: what in theory we refer to as the *reference* signal) **is higher** than the **process variable** (**PV**: the output temperature as sensed by the RTD, and fed back to the controller).
2. If the **SV** is set to a value less than the **PV**, the **Microcontrol X** actually turns off, and a fan provided on the oven turns on. It remains operational until the **PV** drops to the **SV** once again, when the **Microcontrol X** action on the heater becomes operational again.
3. When the SCADA is connected to the controller (as it is in all your experiments), the settings you make on the SCADA panel are automatically updated in the Fuji controller, that is, the SCADA has a higher priority to settings you separately make on the Fuji controller. Similarly, PV sensed by the Fuji controller is automatically updated on the SCADA screen, that is, sensed variables from the controller have a higher priority to those on the SCADA panel.
4. The controller constants the that you set on the SCADA screen, and variables that are displayed on it are all analog. Since the control is actually digital, the conversion of all numbers to equivalent digital ones is always kept internal to commercial controllers. This is partly to maintain confidentiality of design, and partly to make the operation robust against inadvertent tampering.
5. Finally, since a SCADA system always plots *real-time signals*, remember that *the time always increases to the left of the axis* ! That is, the current instant of all plots (at the extreme left) is always higher than all previous instances.

³ **SCADA: System Control And Data Acquisition**; one of the well accepted platform types for real time implementation of digital control worldwide.

Tutorial:

Check out the following webpages to help you with understanding the commercial products involved in the experiment:

- <https://www.instrumart.com/assets/PXF5-PXF9-manual.pdf>
- <https://support.industry.siemens.com/cs/document/109752566/simatic-step-7-and-wincc-v15-trial-download-?dti=0&lc=en-WW>
- [https://www.automationdirect.com/adc/overview/catalog/process_control -a- measurement/temperature sensors -a- transmitters/rtd \(pt100\) sensors](https://www.automationdirect.com/adc/overview/catalog/process_control/-a-measurement-temperature-sensors-a-transmitters/rtd-pt100-sensors)

Project:

Since the ovens are all hooked up in closed loop (and you are not allowed to open the loops !) We will find an alternate way to obtain the system time constants.

- On the SCADA screen, set the proportional constant to a very low value, with the other two constants set to zero.
- For the set value of K_p (known, of course), derive the CLTF of a first order system with OL gain of unity (only the time constant should be unknown).
- Apply a step change to **SV**, and record different values of **PV** as time progresses, until the two become equal.
- From the first order response of the CLTF with known K_p , compute the oven system time constant.
- Repeat for a few different K_p settings, to get multiple estimates of the oven time constant, and obtain an average from all step change experiments.

Knowing the first order time constant, obtain the parameters (SS gain, delay, and lag) for a lag transfer function in the usual ***ZN open-loop tuning process*** for lag systems.

Hence by ***ZN rules***, obtain the gains for P, PI, and PID controllers. Tabulate your ***ZN designs***.

With your designed controllers in place (programming them through SCADA panel in *Man* mode) create CL controls for the oven. Study the **steady state error**, and **settling time** (select the percent) for different step changes of **SV**.

To have the controller gains set by automated computation internal to the controller, change *Man* to *Auto* on the SCADA panel. This change takes the designed controllers out of the CL, and inserts the control constants as computed internally. Once again, study the **steady state error**, and **settling time** (select the percent) for same step changes of **SV** as used earlier.

For observations and discussions:

For both experiments, tabulate your Ziegler-Nichols controller designs. Discuss the performance of the oven with the controllers in place, comparing the performance with that of the controllers internally computed.

Discuss various practical aspects of the results, and influencing factors that you observe for both problems.

Experiment 7

Controller design on MATLAB platform using discrete root loci.

Background:

Recall that by the nature of *magnitude* and *phase angle relations* that apply to CL characteristic equations, the *CL poles follow similar trends with gain variations* regardless of the domain being *analog* or *digital*. Their interpretation in the two domains differ, essentially because the *constant ω_n* and *constant- ζ* lines differ between them.

Precisely due to this reason, the functions relevant to the present experiment are mostly the same as those for Expt. 5; the only difference being between the use of `zgrid` and `sgrid`, that allows the difference in interpretations.

Objective:

This project therefore requires design of a cascade feedback controller for a given digital transfer function, according to desired specifications.

Any effect of the selected sampling time needs to be carefully worked out..

(The key to the experiment is obviously: *correct interpretation of the plots* , which is considerably more involved in the present case !)

Tutorial: (to be covered in not more than an hour)

In the MATLAB platform, go through the procedural steps as described for:

- *Root locus design:* <https://www.mathworks.com/help/control/ug/root-locus-design.html>

Elements of the software for familiarisation:

- App:** `controlSystemDesigner('rlocus','*')`
- Functions/keywords:** `rlocus`; `rlocusplot`; `getoptions`; `setoptions`; `zgrid`; `plotoptions`; `tf`; `zpk`.

Project: (to be covered in about two lab hours or with additional study hours, if required.)

For a second order analog system, when closed with a unity negative feedback through a proportional analog controller, the performance requirements in terms of step response are:

- 5% or less peak overshoot, and
- 9s or less settling time.

The digitised transfer function for 1s sampling time is provided to you.

Obtain the value of *digital proportional control gain*, that would make the CLTF meet the two analog performance requirements simultaneously.

Repeat the exercise at different sampling times from 0.1s to 1s, in steps of 0.1s. Remember to change the transfer function poles, zeros, and gains with every sampling time !

For observations and discussions:

To what extent can the *analog performance requirements* be met by *digital proportional control* ?

Are there any restrictions to the above ?

Assuming that the performance criteria can be met, how does the *proportional control gain* requirement change with the sampling time ?

Experiment 9

Controller design on MATLAB platform by discrete frequency response.

Background:

Frequency response plots in MATLAB have no fundamental difference of usage between the analog and digital domains. Once the transfer function is defined in terms of three parameters (*numerator coefficients array, denominator coefficients array, time step*), it is automatically taken to be a digital one by virtue of the third parameter. Once this is done, a frequency response functions always yield digital domain plots of the main band with the Nyquist sampling frequency marked at the band limit.

In Expt. 5, an analog system controller was designed for a stated phase margin requirement. In the present experiment we check out the impact of gain and sampling time on digital gain and phase margins..

Objective:

For a digital system that may be implementable by a processor in real time, we examine the extent to which it is *susceptible to aliasing* at band boundaries.

Further, *design of low order antialiasing filters* is attempted so as to *eliminate aliasing effect* for different sampling times.

Tutorial: (to be covered in not more than an hour)

In the MATLAB platform, go through the procedural steps as described for:

- *Frequency-Domain Characteristics on Response Plots:*
<https://www.mathworks.com/help/control/ug/system-characteristics-on-response-plots.html>
- *Frequency response of a SISO system:*
<https://www.mathworks.com/help/control/ug/frequency-response-of-a-iso-system.html>
- *Creating Discrete-Time Models:*
<https://in.mathworks.com/help/control/examples/creating-discrete-time-models.html>

Elements of the software for familiarisation:

a. **Functions/keywords:** bode; bodeoptions; bodeplot; freqresp; sigma; sigmaplot; tf; zpk.

b. **Tools:** Plot customization:
<https://www.mathworks.com/help/control/analysis-plot-customization.html>

Project: (to be covered in about two lab hours or with additional study hours, if required.)

The closed loop characteristic polynomial of a digital process, implementable in real time, is provided but with no information on the *sampling time* T used for realisation !

As a first step, assume the sampling time to be 1s, plot the frequency response. If the response indicates aliasing at the band boundaries, then clearly the 1s sampling time will not do for the process.

Repeat the exercise for different choice of time step, in *attempt to eliminate aliasing* at band boundaries. Check if this can be achieved perfectly ?

For each value of sampling time that indicates aliasing at band boundaries, cascade the OLTF with *appropriate low pass transfer functions* in attempt to eliminate aliasing at boundaries. As you may be able to reason out, such anti-aliasing filters should preferably have the following features:

- ♦ *Low pass* of low order,
- ♦ Unity DC gain (why is this feature to be preferred ?)
- ♦ Corner frequency such that a significant drop in magnitude eliminates aliasing close to the band boundary.

For observations and discussions:

Record and discuss *aliasing at band boundaries*, and possible elimination by

- a. Appropriate choice of sampling time.
- b. Inclusion of anti-aliasing filters.

Include all frequency response, graphs, and plots required to substantiate your observations and conclusions.

Experiment 10

Performance of a stepper motor as servo and drive actuator.

Background:

When the electric motor is used as an **actuator** (generator of a *control output*, that is, the signal u in a control system), the control output is either in the form of *rotary position* or in the form of a *rotary speed*. Accordingly, the motor is said to be employed as a *servo* or a *drive actuator*, respectively.⁴

Stepper motors (which were originally developed by some German engineers around the time of WW-II), are totally tailored to *servo applications*, in the sense that they *digitise the position* that is to be controlled. Generally slow devices in terms of speed, their performance may not be as good as drive motors in terms of acceleration, deceleration – certain designs may be incapable of operating at all speeds !

There are two fundamental designs of stepper motors, though hybrids of the two are popular with various manufacturers:

1. *Variable reluctance structures* have some integer number N_s equally spaced salient poles on the stator, each provided with an identical DC excitation winding. The rotor (typically low speeds of operation) has some N_r salient poles (not the same as N_s), but without any winding or magnet on them. As each pole winding is excited with DC current sequentially, the rotor pole that is nearest to an excited stator pole aligns with it due to *reluctance torque*. With the sequential excitation continuing over time (may or may not be at uniform speed), *each rotor pole* at some position or the other aligns with *each stator pole*, that is, we have $N_s \times N_r$ steps over 360° rotation.
2. *Permanent magnet structures* have a stator similar to the above, but a cylindrical rotor (typically high speeds of operation) with N_r permanent magnet poles on rotor – N_r obviously as an even number by necessity. Once again, we realise $N_s \times N_r$ steps over 360° rotation.

STEP	PH-1 (U3)	PH-2 (U1)	A-2 (U2)	B-2 (U4)
1	1	0	1	0
2	0	1	1	0
3	0	1	0	1
4	1	0	0	1

⁴ While it is obvious that all motors must physically be associate with both variables: *speed* and *angle*; they differ by design according to *the actuator signal* of interest. Thus motors intended for *servo applications* (eg. control of valve openings, robot joints, etc.) are designed for *accurate positioning of the rotor* but with lesser perfection in terms of *running performance*. Motors designed for *drive applications* (eg. pumps, mill rollers, etc.) are better designed for continuous run, acceleration/deceleration, etc. with practically no check on positions of rest !

For both cases above, as well as for derived hybrid structures, the angle $\theta_s = 360^\circ/(N_s.N_r)$ represents the discretisation of position, and is defined as the *step angle* of the motor.

In this experiment we are provided with a 1.8° stepper motor, complete with controlling IC's in the form of a study kit. Precise specifications are as follows:

- Permanent magnet DC stepper motor with *two phase bifilar*⁵ wound stator.
- Step angle : $1.8^\circ \pm 5\%$ *non-cumulative*.
- Steps/revolutions: 200.

(Check out the interpretation of the specification terms that are in *italics*.)

Objective:

Performance evaluation of a 1.8° stepper motor for *servo* and *drive* operations.

Tutorial:

For better feel of the operation of steppers, you may visit several of the available websites, such as:

- <https://www.engineersgarage.com/articles/stepper-motors>, or
- <https://www.elprocus.com/stepper-motor-types-advantages-applications/>.

Project:

(For every change of mode, always power on again in order to activate the new mode.)

Servo operation:

1. Set the switch **SW3** to *Step*.
2. Power on the setup.
3. Position the **Mode** switch to *Automatic* (low position). Press the **RESET** button.
4. Enter the *number of steps* intended for the servo to travel by pressing **UP** button (the number increases progressively, and decides the total steps that the rotor can be pulsed to travel before it becomes stationary).
5. Position the **Mode** switch to *Manual*.
6. Pulse the motor to step ahead by repeatedly pressing **CN1**.
7. Observe the rotation in forward direction by the number of steps programmed, before the rotor stops altogether.
8. Set the **Mode** switch to *Automatic*. Press the **RESET** button.
9. Enter the *negative number of steps* by pressing **DOWN** button.
10. Position the **Mode** switch to *Manual*.

⁵ A *bifilar* winding on a stator pole is a combo of *two windings intertwined* on each pole; the two connected in a manner so as to carry current in opposite direction. This allows rapid switch between north and south oriented flux at each pole: something that can be an asset to stepper operation, which is always required to be as fast as possible.

11. Pulse the motor by **CN1** and observe the motor stepping in reverse direction up to number of steps programmed.

The switching sequence of the stator phases (as indicated by the blinking LED's U3-U1-U2-U4 provided at the head of the kit) is as follows:⁶

Drive operation:

1. Set the switch **SW3** to *Free Run*.
2. Set the switch **SW1** to *clockwise* or *anti-clockwise* direction.
3. Power on the setup.
4. Position the **Mode** switch to *Manual*. Press the **RESET** button.
5. The *speed control potentiometer* (look for a knob provided) can be used to run the motor as a drive by fast changing excitation of stator windings.

For observations and discussions:

Servo operation:

- Starting with the **rest position** that you precisely choose on the circular scale on the motor shaft, obtain the *number of steps* (in terms of **CN1** pulses) required to be set (*Servo operation*, step #4) to reach up to *successive angular graduations* on the circular scale.
- Some trial-and-error may be required for *number of steps* setting corresponding to *each servo position* to be reached from the **rest position** (*rest-to-first*, *rest-to-second*, etc.), until you cover *half a revolution* in either direction ($\pm 180^\circ$ revolution).
- Tabulate the *angle for each servo position* (all positions together cover 360° ; accordingly the degree equivalent of each position can be found).
- The *number of steps* setting for each servo position (as obtained from the digital display) may be recorded. These may be multiplied by the stepping angle of 1.8° to find the *expected angle of travel* for each servo position (from rest position).
- Compute the error between *expected angle of travel* and *actual angle for each servo position*. Discuss the extent of the error, together with the factors that may contribute to them in a cumulative manner.

Drive operation:

- Set the speed control potentiometer (*Drive operation*, step #5) at a low speed, so that the cycles of LED indicating lights can be counted.
- The four LED patterns indicate four steps (check the above table). Using the timer on your mobile phone, time a number of LED patterns (say, a multiple of four). Since the LED's are triggered as the rotor changes positions, (*no. of steps timed* $\times 1.8^\circ$) is the total angle of rotation corresponding to the time that you record. Hence calculate the angular speed.
- For each reading above, obtain the number on digital display (the *electrical number of steps*, by pulses), as calculated.
- Tabulate the two results, together with the error between them that you may discuss.

⁶ The above switching sequence/logic will move shaft in forward direction. For reverse direction, read the table upward.

- In a separate part of the experiment, set the potentiometer to the *maximum possible frequency*. At this rate you will not be able to count the LED blinks, but the rate is specified as 10000 steps/s (10000 Hz).
- Count the *number of steps* you record on display corresponding to different *lengths of time* recorded on your timer (mobile). Hence calculate the stepper speed, and compare it to the figure of 10000 Hz. Discuss the error, if any.
- The same experiment can be conducted at say, half the setting of the potentiometer, quarter setting of the potentiometer, etc. right down to low speeds as described in the earlier experiment.

Experiment 11

State feedback controller design on MATLAB platform.

Background:

Because of its excessive dependence on matrix algebra and vector spaces, MATLAB functions that are useful for state space control are somewhat scattered between the **control systems** toolbox and the **symbolic math** toolbox. Here we will try to put together the functions that are of primary use to us for time domain (that is, eigenvalue based) control designs.

There can be alternative approaches to state space based control design – topics that are beyond the scope of the present course.

Objective:

To design the state feedback gain matrix for a given analog state-space system to satisfy required performance specifications..

Tutorial: (to be covered in not more than an hour)

In the MATLAB platform, go through the procedural steps as described for:

- *Create state-space model:* <https://in.mathworks.com/help/control/ref/ss.html>
- *Eigenvalues and Eigenvectors:*
https://in.mathworks.com/help/symbolic/eigenvalues-and-eigenvectors.html?searchHighlight=eigenvalues&s_tid=doc_srchtile
- *Creating Discrete-Time Models:*
<https://in.mathworks.com/help/control/examples/creating-discrete-time-models.html>
- *Pole placement design:*
https://in.mathworks.com/help/control/ref/place.html?searchHighlight=place&s_tid=doc_srchtile
- *Observability:* <https://in.mathworks.com/help/control/ref/obsv.html>
- *Controllability:* <https://in.mathworks.com/help/control/ref/ctrb.html>

Elements of the software for familiarisation:

- a. **Functions/keywords:** ctrb; obsv; eigenvalues; eigenvectors; jordanform; place; ss.
- b. **Tools:** Symbolic math toolbox:
https://in.mathworks.com/help/symbolic/index.html?searchHighlight=symbolic%20math&s_tid=doc_srchtile

Project: (to be covered in about two lab hours or with additional study hours, if required.)

The third order digital state-space specified, is accompanied by a *nominal set* of CLTF eigenvalues when fitted with a positive state feedback controller.

State feedback gain matrices are to be designed so that:

- *Maximum magnitude* of all eigenvalues are as in the *nominal set*.
- The CL system is *always observable* and *controllable*.

For observations and discussions:

Record and discuss the combination of feedback gains, showing how they ensure the performance required as above.

In situations where the no designed value of the gain matrix is possible, explain and discuss why this is so.

Experiment 12

Nonlinear system dynamics on Simulink for different Lyapunov control designs.

Background:

The following differential equations represent the nonlinear system for a *pendulum with friction*, and accommodating *large angular displacements*, to be stabilised by a Lyapunov controller with different energy components.:

$$\begin{aligned}\dot{x}_1(t) &= x_2(t) \\ \dot{x}_2(t) &= -\frac{g}{L} \cdot \sin x_1(t) - \frac{k}{m} \cdot x_2(t) + \frac{1}{mL^2} \cdot T(t)\end{aligned}$$

Variables for the problem include:

Input:

T : Torque in N-m, applied to the pendulum mass.

Output:

x_1 : Angular position of pendulum mass in rads.

States:

x_1 : Angular position of pendulum mass in rads.

x_2 : Angular speed of pendulum mass in rads/s.

The parameter m can be assigned a nominal value of 1kg, while L can be similarly assigned a nominal value of 1m. The friction constant k can be assigned a value of 0.01 N/ms⁻¹. All three can be assigned different values for more exhaustive studies.

The parameter g is constant at 9.8m/s².

Objective:

Dynamic studies of the pendulum described above, with which different Lyapunov control designs are to be incorporated.

The systems are to be individually simulated on Simulink; each using a detailed nonlinear state space system simulation in two state variables..

Remember that your major aim is to keep the mass stable at any equilibrium point !

Tutorial:

Brush up your Lyapunov design concepts very thoroughly for this one.

Further, rehash the Simulink Primer in your Lab account, unless already done so.

Project:

Consider two different energy-based function components, additive combinations of which can be used as different Lyapunov functions:

A. Proportionate to *square of potential energy* : $= K_{PE}^l \cdot (x_{1,ref} - x_1)^2$

B. Proportionate to *kinetic energy* : $= K_{KE}^l \cdot x_2^2$

where the design problem amounts to determination of K -constants by **Lyapunov's Second Method** (which is the formal name for what we have been referring to as "Lyapunov control design").

- For different control designs, Lyapunov functions can be generated by one or both of "A" and "B". Accordingly, Lyapunov control gains will evolve.
- Be careful, because not all control logics may be equally effective !
- Note that as in Physics concepts, *potential energy* is proportionate to *displacement*. Function component "A" has been defined as square of these, simply for ease of differentiability. *Kinetic energy* is proportionate to *square of speed*, so it have been retained without squaring.

Create combinations of components for appropriate Lyapunov functions, and design control by each.

For observations and discussions:

Using step, impulse, and ramp inputs for $x_{1, ref}$ (the reference position of the pendulum mass), study dynamics of the system from one equilibrium to another, using traces of both state variables as well as the single output variable.

The studies should be done for each Lyapunov control design you work out. Accordingly, conclude the best Lyapunov control for the nonlinear system to work with (you may provide a ranking between the various controls that you find to be effective). You are expected to carefully reason out your preferences.

*Make sure your submission includes the **detailed patched block diagram** for the overall system from the Simulink main screen. A brief write-up on the simulation, involving notes and comments, could add credit to your work.*